

Markov Decision Process and Reinforcement Learning

Types of Learning

- Supervised Learning

- A situation in which sample (input, output) pairs of the function to be learned can be perceived or are given
- You can think about it as if there is a kind teacher
 - Training data: (X,Y) . (features, label)
 - Predict Y , minimizing some loss.
 - Regression, Classification.

- Unsupervised Learning

- Training data: X . (features only)
- Find “similar” points in high-dim X -space.
- Clustering.

Example of Supervised Learning

- Predict the price of a stock in 6 months from now, based on economic data. (Regression)
- Predict whether a patient, hospitalized due to a heart attack, will have a second heart attack. The prediction is to be based on demographic, diet and clinical measurements for that patient. (Logistic Regression)
- Identify the numbers in a handwritten ZIP code, from a digitized image (pixels). (Classification)

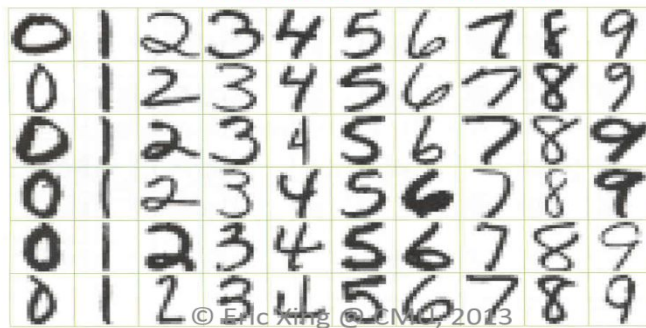
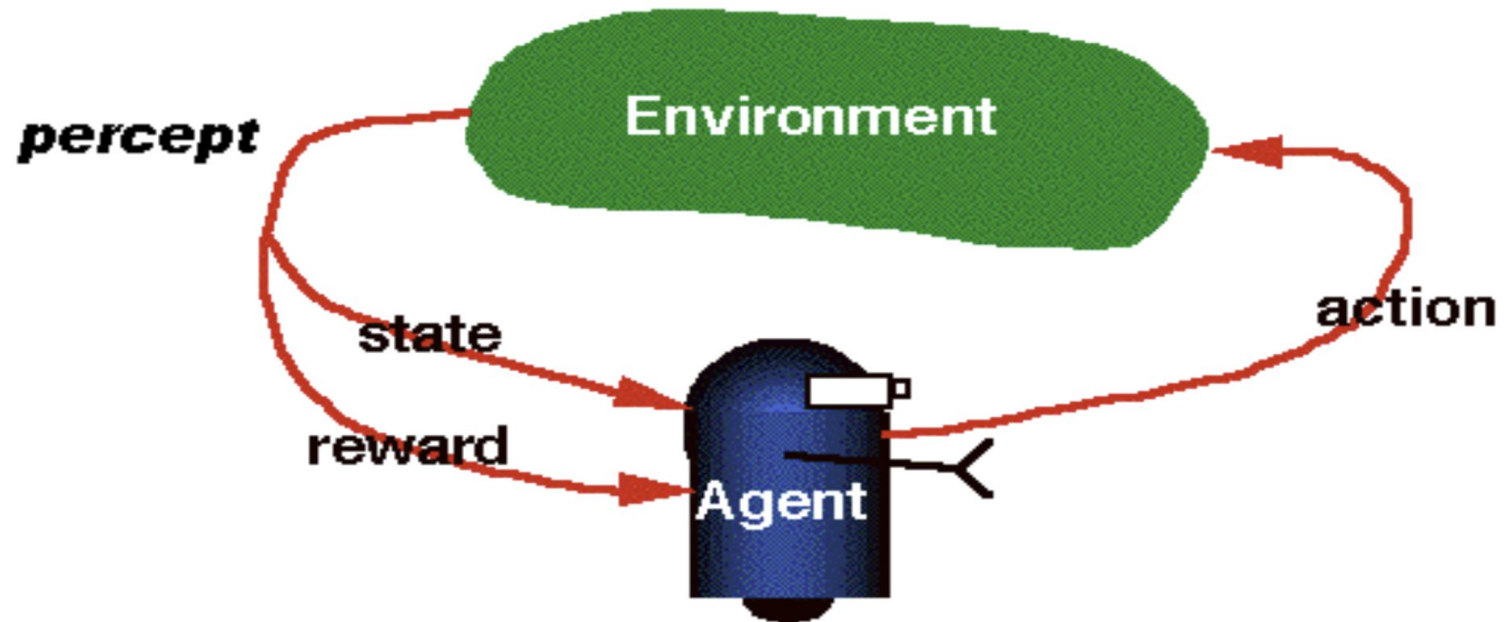


FIGURE 1.2. Examples of handwritten digits from U.S. postal envelopes.

What is Learning?

- Learning takes place as a result of interaction between an **agent** and the **world**. The idea behind learning is that
 - Percepts received by an agent should be used not only for understanding/interpreting/prediction, as in the machine learning tasks we have addressed so far, but also for **acting**, and furthermore for improving the agent's ability to behave optimally in the future to achieve the goal.

RL is learning from interaction



Examples of Reinforcement Learning

- How should a robot behave so as to optimize its “performance”? (Robotics)



- How to automate the motion of a helicopter? (Control Theory)



- How to make a good chess-playing program? (Artificial Intelligence)



Supervised Learning vs Reinforcement Learning

- Supervised learning

$$f : X \rightarrow Y$$

- X: inputs
- Y: outputs
- The predicted outputs can be evaluated immediately by the teacher
- f evaluated with loss function

- Reinforcement learning for learning a policy

$$\pi : S \mapsto A$$

- S: states
- A: actions
- Take action A to affect state S
- The predicted action A cannot be evaluated directly but can be given positive/negative rewards
- Policy evaluated with value functions

Reinforcement Learning

- Reinforcement Learning
 - in the case of the agent acting on its environment, it receives some evaluation of its action (reinforcement), but is not told about which action is the correct one to achieve its goal
 - Imagine learning to go back to GHC from this room ...
 - Training data: (S, A, R). (State-Action-Reward)
 - Develop an optimal **policy** (sequence of decision rules) for the learner so as to maximize its **long-term reward**.
 - **Robotics, Board game playing programs.**

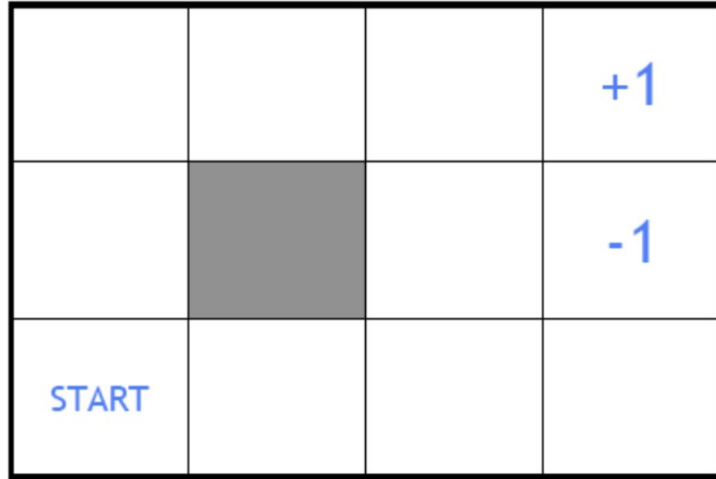
Elements of RL

- **policy** $\pi : S \mapsto A$
 - A map from **state space** to **action space**.
 - May be stochastic.
- **reward function** $R(S)$
 - It maps each state (or, state-action pair) to a real number, called **reward**.
- **value function**
 - Value of a state (or, state-action pair) is the **total expected reward**, starting from that state (or, state-action pair).

History of Reinforcement Learning

- Roots in the **psychology of animal learning** (**Thorndike, 1911**).
- Another independent thread was the problem of **optimal control**, and its solution using **dynamic programming** (**Bellman, 1957**).
- Idea of **temporal difference** learning (on-line method), e.g., playing board games (**Samuel, 1959**).
- A major breakthrough was the discovery of **Q-learning** (**Watkins, 1989**).

Robot in a room



actions: UP, DOWN, LEFT, RIGHT

UP

80%

move UP

10%

move LEFT

10%

move RIGHT



- reward +1 at [4,3], -1 at [4,2]
- reward -0.04 for each step
- what's the strategy to achieve max reward?
- what if the actions were NOT deterministic?

The Precise Goal

- To find a **policy** $\pi : S \mapsto A$ that maximizes the **Value function**.
- There are different approaches to achieve this goal in various situations.
- **Value iteration** and **Policy iteration** are two more classic approaches to this problem. But essentially both are **dynamic programming**.
- **Q-learning** is a more recent approaches to this problem. Essentially it is a **temporal-difference method**.

Markov Decision Processes

A Markov decision process is a tuple $(S, A, \{P_{sa}\}, \gamma, R)$ where:

- S is a set of **states**. (For example, in autonomous helicopter flight, S might be the set of all possible positions and orientations of the helicopter.)
- A is a set of **actions**. (For example, the set of all possible directions in which you can push the helicopter's control sticks.)
- P_{sa} are the state transition probabilities. For each state $s \in S$ and action $a \in A$, P_{sa} is a distribution over the state space. We'll say more about this later, but briefly, P_{sa} gives the distribution over what states we will transition to if we take action a in state s .
- $\gamma \in [0, 1)$ is called the **discount factor**.
- $R : S \times A \mapsto \mathbb{R}$ is the **reward function**. (Rewards are sometimes also written as a function of a state S only, in which case we would have $R : S \mapsto \mathbb{R}$).

The dynamics of an MDP

- We start in some state s_0 , and get to choose some action $a_0 \in A$
- As a result of our choice, the state of the MDP randomly transitions to some successor state s_1 , drawn according to $s_1 \sim P_{s_0 a_0}$
- Then, we get to pick another action a_1
- ...

$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} s_3 \xrightarrow{a_3} \dots$$

The dynamics of an MDP, (Cont'd)

- Upon visiting the sequence of states $s_0, s_1, \dots$, with actions $a_0, a_1, \dots$, our total payoff is given by

$$R(s_0, a_0) + \gamma R(s_1, a_1) + \gamma^2 R(s_2, a_2) + \dots$$

- Or, when we are writing rewards as a function of the states only, this becomes

$$R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots$$

- For most of our development, we will use the simpler state-rewards $R(s)$, though the generalization to state-action rewards $R(s; a)$ offers no special difficulties.
- Our goal in reinforcement learning is to choose actions over time so as to maximize the expected value of the total payoff:

$$\mathbb{E}[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots]$$

Policy

- A policy is any function $\pi : S \mapsto A$ mapping from the states to the actions.
- We say that we are executing some policy if, whenever we are in state s , we take action $a = \pi(s)$.
- We also define the value function for a policy π according to

$$V^\pi(s) = \mathbb{E}[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots \mid s_0 = s, \pi]$$

- $V^\pi(s)$ is simply the expected sum of discounted rewards upon starting in state s , and taking actions according to π .

Value Function

- Given a fixed policy π , its value function V^π satisfies the **Bellman equations**:

$$V^\pi(s) = R(s) + \gamma \sum_{s' \in \mathcal{S}} P_{s\pi(s)}(s') V^\pi(s')$$

Immediate reward

expected sum of future discounted rewards

- Bellman's equations can be used to efficiently solve for V^π (see later)

